AmanAI Lab — Agentic AI Interview Prep Series Day 1 Complete Notes — **Memory & Tools**, Concepts & Real Interview Questions

Series — Agentic AI Interview Prep | Day 2 of 5 | @AmanAI_lab

1. How would you design memory for an AI agent that runs across multiple user sessions?

☑ 1. Big Picture (Start like this in video)

"An AI agent has two types of memory:

- **Short-term memory → current conversation**
- **Long-term memory → stored data across sessions**

The key difference is:
👉 *Where the data is stored and how long it lives*"

◇ 2. Short-Term Memory (Where is it stored?)

👉 Simple Answer:

Short-term memory is stored **inside the LLM prompt (context window)**

📌 What does that mean?

- Every time you send a message to an LLM (like GPT), you also send:
  - Previous messages
  - System instructions
- This full input = **context**

➡ This is the **temporary memory**

📌 Where exactly is it stored?

- Not in a database ❌
- Not permanently ❌
- It exists only:
  👉 In **RAM / API request payload** during execution

📌 **Example**

```
User: I want a laptop under 80k
Assistant: Suggests options
User: It should be lightweight
```

➡️ When sending 2nd query, system sends:

```
[Previous messages + new message]
```

➡️ That becomes **short-term memory**

⚠️ **Limitation**

- Limited by **context window (e.g., 8k / 32k tokens)**
- Lost when session ends

🔷 **3. Long-Term Memory (Where is it stored?)**

👉 **Simple Answer:**

Long-term memory is stored in **databases**

📌 **Types of Storage**

🟣 **(A) Vector Database (MOST IMPORTANT 🔥)**

Used for:

- Semantic search (meaning-based)
- Storing embeddings

📌 Examples:

- FAISS
- Pinecone
- Weaviate
- Chroma

🟣 **(B) Traditional Database**

Used for:

- Structured data

📌 Examples:

- PostgreSQL
- MySQL
- MongoDB

📌 What do we store?

- User preferences
- Past interactions
- Important facts

📌 Example (Very Important)

User says:
👉 "I prefer Apple products and budget is 80k"

Stored as:

In PostgreSQL (structured):

```
user_id | preference | budget
101     | Apple      | 80000
```

In Vector DB (semantic):

```
Text: "User prefers Apple products under 80k"
→ Converted into embedding → stored
```

🔷 4. How Retrieval Works (Core Concept 🔥)

When user comes again:

User: "Suggest a laptop"

Step-by-step:

1. Convert query → embedding
2. Search in vector DB
3. Retrieve similar past memory
4. Inject into LLM prompt

📌 Final Prompt Looks Like:

```
User preference: Apple, budget 80k
User query: Suggest laptop
```

➡️ Now LLM gives **personalized answer**

🔷 5. Full Flow (Very Important)

1. User interacts
2. Important data extracted
3. Stored in DB (PostgreSQL + Vector DB)
4. New session starts
5. Retrieve relevant memory
6. Inject into prompt
7. Generate response

🔷 6. Real-Life Analogy (Best for Video)

🧠 Human Brain Example:

- Short-term memory → What you are thinking right now
- Long-term memory → Your past experiences

➡️ You don't remember everything, only **relevant things when needed**

🔷 7. Common Mistakes (Say this in video)

❌ Storing full conversation blindly
❌ Not filtering memory
❌ Retrieving too much irrelevant data

🔷 8. Final One-Line Summary

👉 "Short-term memory lives inside the prompt,
and long-term memory lives inside databases like vector DBs and PostgreSQL."

🔥 Pro Tip (Important for your level)

If interviewer asks:
👉 "Where exactly memory is stored?"

Answer confidently:

- Short-term → **Prompt / Context Window (RAM)**
- Long-term → **Vector DB + SQL/NoSQL DB**

2. How does tool calling work internally in an LLM-based agent?

> "Tool calling means the LLM can **decide to use an external function/API** instead of answering directly, to get accurate or real-time information."

◇ 3. Why Tool Calling is Needed?

LLMs have limitations:

- ❌ No real-time data
- ❌ Cannot perform actions (book ticket, fetch DB)
- ❌ Can hallucinate

👉 Tools solve this:

- APIs
- Databases
- Calculators
- External services

◇ 4. Where are Tools Stored?

👉 Tools are NOT inside LLM

They are defined in your **backend system**

📌 Example Tool Definition (Python)

```
def get_weather(city: str) -> str:
    return f"Weather in {city} is 30°C"
```

📌 How LLM sees this?

You pass tool schema in API:

```
{
  "name": "get_weather",
```

```
  "description": "Get weather of a city",
  "parameters": {
    "city": "string"
  }
}
```

👉 This schema is sent in **API request payload**

🔷 5. Step-by-Step Internal Flow (Very Important 🔥)

🧠 Step 1: User Input

User:
👉 “What is the weather in Delhi?”

🧠 Step 2: LLM Decision

LLM analyzes:

- Can I answer? ❌
- Do I need a tool? ☑

➡ Decides to call tool

🧠 Step 3: Tool Call Generation

LLM outputs (structured):

```
{
  "tool": "get_weather",
  "arguments": {
    "city": "Delhi"
  }
}
```

🧠 Step 4: Backend Executes Tool

Your backend:

- Receives tool call
- Runs actual function/API

```
get_weather("Delhi")
```

🧠 Step 5: Tool Output

```
"Weather in Delhi is 30°C"
```

🧠 Step 6: Send Back to LLM

Now backend sends:

```
{
  "tool_result": "Weather in Delhi is 30°C"
}
```

🧠 Step 7: Final Response Generation

LLM combines:

- User query
- Tool result

👉 Final answer:

“Currently, the weather in Delhi is 30°C.”

◇ 6. Where Everything Happens (Important 🔥)

Component	Where it lives
LLM	API (OpenAI, Claude, etc.)
Tools	Your backend (Python/Node)
Tool schema	API request payload
Execution	Backend server
Result	Sent back to LLM

◇ 7. Real-World Example

🛒 E-commerce Agent

User:
👉 “What is the price of iPhone 15?”

Flow:

1. LLM decides → needs DB
2. Calls tool → `get_product_price`
3. Backend → queries database
4. Returns → ₹75,000
5. LLM → final answer

◇ 8. Key Concepts (Important Keywords)

- Function Calling
- API Integration
- Structured Output (JSON)
- Agent Loop (Think → Act → Observe)

◇ 9. Common Problems

- ❌ Wrong tool selection
- ❌ Hallucinated tool calls
- ❌ Invalid arguments
- ❌ Tool failure / timeout

◇ 10. Solutions

- ✔ Clear tool descriptions
- ✔ Validation layer
- ✔ Retry mechanism
- ✔ Error handling

◇ 11. Final One-Line Summary

👉 "Tool calling allows an LLM to move from just generating text to actually **taking actions using external systems**."

🔥 Pro Tip (Interview Level)

If interviewer asks:
👉 "Where does tool calling happen?"

Answer:

- Decision → inside LLM

- Execution → backend system
- Communication → API payload